Trabajo Practico SQL

Unidad 2 – Base de Datos

A) Se tiene la siguiente base de datos relacional:

Documentos (cod_documento, descripción)

Oficinas (cod_oficina, codigo_director, descripcion)

Empleados (cod_empleado, apellido, nombre, fecha_nacimiento, num_doc, cod_jefe, cod_oficina, cod_documento)

Datos_contratos (cod_empleado, fecha_contrato, cuota, ventas)

Fabricantes (cod_fabricante, razón_social)

Listas (cod_lista, descripción, ganancia)

Productos (cod_producto, descripcion, precio, cantidad_stock, punto_reposición, cod_fabricante)

Precios (cod_producto, cod_lista, precio)

Clientes (cod_cliente, cod_lista, razón_social)

Pedidos (cod_pedido, fecha_pedido, cod_empleado, cod_cliente)

Detalle_pedidos (cod_pedido, numero_linea, cod_producto, cantidad)

Resolver las siguientes consultas utilizando sentencias SQL:

• **Consultas simples** (una sola tabla)

1) Obtener una lista con los nombres de las distintas oficinas de la empresa.

```
SELECT descripcion FROM Oficinas;
```

2) Obtener una lista de todos los productos indicando descripción del producto, su precio de costo y su precio de costo IVA incluído (tomar el IVA como 21%).

```
SELECT descripcion, precio, precio * 1.21 AS precio_con_iva

FROM Productos;
```

3) Obtener una lista indicando para cada empleado apellido, nombre, fecha de cumpleaños y edad.

```
SELECT apellido, nombre,

FORMAT(fecha_nacimiento, 'dd/MM') AS cumpleanos,

DATEDIFF(YEAR, fecha_nacimiento, GETDATE()) AS edad

FROM Empleados;
```

4) Listar todos los empleados que tiene un jefe asignado.

```
SELECT * FROM Empleados

WHERE cod_jefe IS NOT NULL;
```

5) Listar los empleados de nombre “María” ordenado por apellido.

```
SELECT *

FROM Empleados

WHERE nombre = 'María'

ORDER BY apellido;
```

6) Listar los clientes cuya razón social comience con “L” ordenado por código de cliente.

```
SELECT *

FROM Clientes

WHERE razón_social LIKE 'L%'

ORDER BY cod_cliente;
```

7) Listar toda la información de los pedidos de Marzo ordenado por fecha de pedido.

```
SELECT *

FROM Pedidos

WHERE MONTH(fecha_pedido) = 3

ORDER BY fecha_pedido;
```

8) Listar las oficinas que no tienen asignado director.

```
SELECT *

FROM Oficinas

WHERE codigo_director IS NULL;
```

9) Listar los 4 productos de menor precio de costo.

```
SELECT TOP 4 * FROM Productos

ORDER BY precio ASC;
```

10) Listar los códigos de empleados de los tres que tengan la mayor cuota.

```
SELECT TOP 3 cod_empleado

FROM Datos_contratos

ORDER BY cuota DESC;
```

• Consultas multitablas

1) De cada producto listar descripción, razón social del fabricante y stock ordenado por razón social y descripción.

```
SELECT p.descripcion, f.razón_social, p.cantidad_stock

FROM Productos p

JOIN Fabricantes f ON p.cod_fabricante = f.cod_fabricante

ORDER BY f.razón_social, p.descripcion;
```

2) De cada pedido listar código de pedido, fecha de pedido, apellido del empleado y razón social del cliente.

```
SELECT p.cod_pedido, p.fecha_pedido, e.apellido, c.razón_social

FROM Pedidos p

JOIN Empleados e ON p.cod_empleado = e.cod_empleado

JOIN Clientes c ON p.cod_cliente = c.cod_cliente;
```

3) Listar por cada empleado apellido, cuota asignada, oficina a la que pertenece ordenado en forma descendente por cuota.

```
SELECT e.apellido, d.cuota, o.descripcion

FROM Empleados e

JOIN Datos_contratos d ON e.cod_empleado = d.cod_empleado

JOIN Oficinas o ON e.cod_oficina = o.cod_oficina

ORDER BY d.cuota DESC;
```

4) Listar sin repetir la razón social de todos aquellos clientes que hicieron pedidos en Abril.

```
SELECT DISTINCT c.razón_social

FROM Clientes c

JOIN Pedidos p ON c.cod_cliente = p.cod_cliente

WHERE MONTH(p.fecha_pedido) = 4;
```

5) Listar sin repetir los productos que fueron pedidos en Marzo.

```
SELECT DISTINCT pr.descripcion

FROM Productos pr

JOIN Detalle_pedidos dp ON pr.cod_producto = dp.cod_producto

JOIN Pedidos p ON dp.cod_pedido = p.cod_pedido

WHERE MONTH(p.fecha_pedido) = 3;
```

6) Listar aquellos empleados que están contratados por más de 10 años ordenado por cantidad de años en forma descendente.

```
SELECT e.apellido, e.nombre, DATEDIFF(YEAR, dc.fecha_contrato, GETDATE()) AS antiguedad

FROM Empleados e

JOIN Datos_contratos dc ON e.cod_empleado = dc.cod_empleado

WHERE DATEDIFF(YEAR, dc.fecha_contrato, GETDATE()) > 10

ORDER BY antiguedad DESC;
```

7) Obtener una lista de los clientes mayoristas ordenada por razón social.

```
SELECT c.cod_cliente,

 c.razón_social

FROM Clientes c

INNER JOIN Listas l ON c.cod_lista = l.cod_lista

WHERE l.descripción LIKE '%Mayorista%'

ORDER BY c.razón_social ASC;
```

8) Obtener una lista sin repetir que indique qué productos compró cada cliente, ordenada por razón social y descripción.

```
CT DISTINCT c.razón_social,

 pr.descripcion AS descripcion_producto

FROM Clientes c

INNER JOIN Pedidos p ON c.cod_cliente = p.cod_cliente

INNER JOIN Detalle_pedidos dp ON p.cod_pedido = dp.cod_pedido

INNER JOIN Productos pr ON dp.cod_producto = pr.cod_producto

ORDER BY c.razón_social ASC, pr.descripcion ASC;
```

9) Obtener una lista con la descripción de aquellos productos cuyo stock está por debajo del punto de reposición indicando cantidad a comprar y razón social del fabricante ordenada por razón social y descripción.

```
SELECT p.descripcion,

 (p.punto_reposición - p.cantidad_stock) AS cantidad_a_comprar,

 f.razón_social

FROM Productos p

INNER JOIN Fabricantes f ON p.cod_fabricante = f.cod_fabricante

WHERE p.cantidad_stock < p.punto_reposición
```

ORDER BY f.razón_social ASC, p.descripcion ASC;

10) Listar aquellos empleados cuya cuota es menor a 50000 o mayor a 100000.

CT e.cod_empleado,

e.apellido,

dc.cuota

FROM Empleados e

INNER JOIN Datos_contratos dc ON e.cod_empleado = dc.cod_empleado

WHERE dc.cuota < 50000 OR dc.cuota > 100000;

B) Dadas las siguientes tablas:

CLIENTES (código_cliente, nombre, domicilio, provincia)

PRODUCTOS (código_producto, nombre_producto)

ITEM_VENTAS (número_factura, coódigo_producto, cantidad, precio)

VENTAS (número_factura, código_cliente, fecha)

Resolver las siguientes consultas SQL:

1) Obtener la cantidad de unidades máxima.

SELECT MAX(cantidad) FROM ITEM_VENTAS;

2) Obtener la cantidad total de unidades vendidas del producto c

SELECT SUM(cantidad) AS Total_Vendidas

FROM ITEM_VENTAS

WHERE código_producto = 'c';

3) Cantidad de unidades vendidas por producto, indicando la descripción del producto, ordenado de mayor a menor por las cantidades vendidas.

SELECT p.nombre_producto, SUM(iv.cantidad) AS total_vendido

FROM PRODUCTOS p

JOIN ITEM_VENTAS iv ON p.código_producto = iv.código_producto

GROUP BY p.nombre_producto

ORDER BY total_vendido DESC;

4) Cantidad de unidades vendidas por producto, indicando la descripción del producto, ordenado alfabéticamente por nombre de producto para los productos que vendieron mas de 30 unidades.

SELECT P.nombre_producto, SUM(IV.cantidad) AS total_unidades

```
FROM PRODUCTOS P

JOIN ITEM_VENTAS IV ON P.código_producto = IV.coódigo_producto

GROUP BY P.código_producto, P.nombre_producto

HAVING SUM(IV.cantidad) > 30

ORDER BY P.nombre_producto ASC;
```

5) Obtener cuantas compras (1 factura = 1 compra) realizó cada cliente indicando el código y nombre del cliente ordenado de mayor a menor

```
SELECT c.código_cliente, c.nombre, COUNT(v.número_factura) AS total_compras

FROM CLIENTES c

JOIN VENTAS v ON c.código_cliente = v.código_cliente

GROUP BY c.código_cliente, c.nombre

ORDER BY total_compras DESC;
```

6) Promedio de unidades vendidas por producto, indicando el código del producto para el cliente 1

```
SELECT IV.coódigo_producto, AVG(IV.cantidad) AS promedio_unidades

FROM ITEM_VENTAS IV

JOIN VENTAS V ON IV.número_factura = V.número_factura

WHERE V.código_cliente = 1

GROUP BY IV.coódigo_producto;
```